Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия

Дисциплина: «Программирование»

Отчёт
По лабораторной работе №3-4
Вариант **112233**

Выполнил:
Крамарь Михаил Романович
P3113

Проверил:
Комлев Игорь Владимирович

г. Санкт-Петербург
2025

Содержание

Текст задания 3
Диаграмма классов объектов 5
Исходный код программы 6
Результат работы программы 7
Промпты для ИИ-ассистентов, примеры полученного кода 9
Выводы по работе 13

Текст задания

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Он встал и подбросил в огонь два больших березовых полена. Когда камин прогорел, в комнате стало темно, и Карлсон зажег керосиновую лампу, которая висела у самого потолка над верстаком. Она осветила теплым, живым светом комнату и все те вещи, которые валялись на верстаке.

Этапы выполнения работы:

1. Получить вариант
2. Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
3. Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
4. Согласовать диаграмму классов и сценарий с преподавателем;
5. Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
6. Продемонстрировать выполнение программы на сервере helios.
7. Ответить на контрольные вопросы и выполнить дополнительное задание.

Текст, выводящийся в результате выполнения программы не обязан дословно повторять текст, полученный в исходном задании. Также не обязательно реализовывать грамматическое согласование форм и падежей слов выводимого текста.

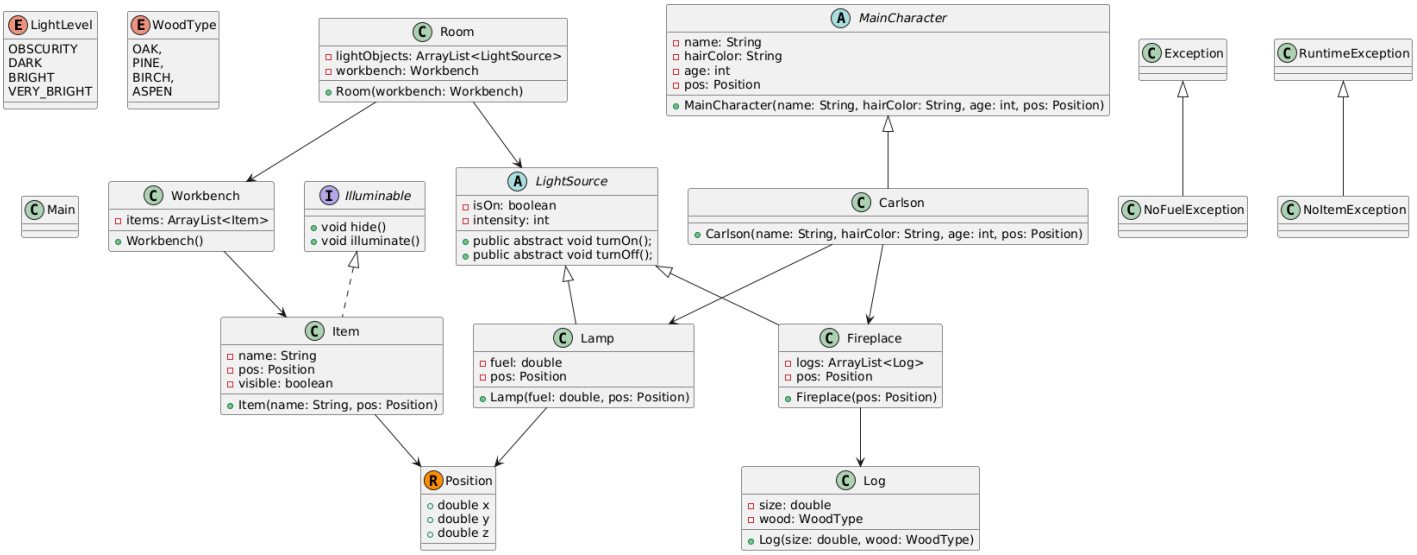
Стоит отметить, что цель разработки объектной модели состоит не в выводе текста, а в эмуляции объектов предметной области, а именно их состояния (поля) и поведения (методы). Методы в разработанных классах должны изменять состояние объектов, а выводимый текст должен являться побочным эффектом, отражающим эти изменения.

Требования к объектной модели, сценарию и программе:

1. В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.

2. Объектная модель должна реализовывать основные принципе ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.
3. Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).
4. Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:
    - абстрактный класс как минимум с одним абстрактным методом;
    - интерфейс;
    - перечисление (enum);
    - запись (record);
    - массив или ArrayList для хранения однотипных объектов;
    - проверяемое исключение.
5. В созданных классах основных персонажей и предметов должны быть корректно переопределены методы equals(), hashCode() и toString(). Для классов-исключений необходимо переопределить метод getMessage().
6. Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно unchecked исключение (можно свое, можно из стандартной библиотеки).
7. При необходимости можно добавить внутренние, локальные и анонимные классы.

Диаграмма классов объектов



Исходный код программы

Ссылка на репозиторий:
https://github.com/moon2light-gif/ITMIUMIU/tree/main/1-ProgrammingJava/Lab3-4/src

Результат работы программы

1. Вариант с включенной лампой:

НЕВЕРОЯТНЫЕ ПРИКЛЮЧЕНИЯ КАРЛСОНА
Текущий уровень освещённости комнаты: OBSCURITY
Текущая позиция Карлсона: 0.0, 0.0, 76.9
Карлсон встал и подошёл к камину, теперь его позиция: 45.0, 10.0, 55.6
Камин до: []
Карслон встал и подбросил в огонь два больших берёзовых полена
Камин после: [Log{size=2.5, wood=BIRCH}, Log{size=2.5, wood=BIRCH}]
Карслон: - Ох, что-то я перетрудился, надо потом тяпнуть варенья
Камин зажжён:
Текущий уровень освещённости комнаты: BRIGHT
предмет Наждачка: освещён, предмет Молоток: освещён, предмет Пила: освещён,
Полено прогорело! Осталось: 1
Полено прогорело! Осталось: 0
Камин потух:
Текущий уровень освещённости комнаты: OBSCURITY
предмет Наждачка: не видим, предмет Молоток: не видим, предмет Пила: не видим,
Керосиновая лампа зажжена:
Карслон: - Ну вот, так гораздо лучше, с таким освещением не грех и ещё раз тяпнуть варенья!
Текущий уровень освещённости комнаты: VERY_BRIGHT
предмет Наждачка: освещён, предмет Молоток: освещён, предмет Пила: освещён,
НУ ВОТ И ЗАКОНЧИЛИСЬ ПРИКЛЮЧЕНИЯ НАШЕГО ГЕРОЯ

2. Вариант с выключенной лампой:

НЕВЕРОЯТНЫЕ ПРИКЛЮЧЕНИЯ КАРЛСОНА
Текущий уровень освещённости комнаты: OBSCURITY
Текущая позиция Карлсона: 0.0, 0.0, 76.9
Карлсон встал и подошёл к камину, теперь его позиция: 45.0, 10.0, 55.6
Камин до: []
Карслон встал и подбросил в огонь два больших берёзовых полена
Камин после: [Log{size=2.5, wood=BIRCH}, Log{size=2.5, wood=BIRCH}]
Карслон: - Ох, что-то я перетрудился, надо потом тяпнуть варенья
Камин зажжён:
Текущий уровень освещённости комнаты: BRIGHT
предмет Наждачка: освещён, предмет Молоток: освещён, предмет Пила: освещён,
Полено прогорело! Осталось: 1
Полено прогорело! Осталось: 0
Камин потух:
Текущий уровень освещённости комнаты: OBSCURITY
предмет Наждачка: не видим, предмет Молоток: не видим, предмет Пила: не видим,

Керосиновая лампа не горит:
Карслон: - Керосин в лампе закончился - она не загорится!!!
Текущий уровень освещённости комнаты: OBSCURITY
предмет Наждачка: не видим, предмет Молоток: не видим, предмет Пила: не видим,
НУ ВОТ И ЗАКОНЧИЛИСЬ ПРИКЛЮЧЕНИЯ НАШЕГО ГЕРОЯ

Промпты для ИИ-ассистентов, примеры полученного кода

Промпт для реализации метода добавления в камин нескольких брёвен:

Смотри, у меня есть класс главного персонажа - Carlson
Нужно реализовать ему метод добавления двух брёвен в камин, камин - это отдельный класс, находящийся в другом пакете
Можешь реализовать этот метод, так чтобы он работал корректно и добавлял в камин два бревна?

1. Реализация метода от Deepseek:

```
import com.house.Fireplace; // импорт класса камина из другого пакета

public class Carlson {
    private String name;

    public Carlson(String name) {
        this.name = name;
    }

    // Метод для добавления двух бревен в камин
    public void addTwoLogsToFireplace(Fireplace fireplace) {
        if (fireplace == null) {
            System.out.println("Ошибка: камин не существует!");
            return;
        }

        System.out.println(name + " добавляет два бревна в камин");
        fireplace.addLog(); // добавляем первое бревно
        fireplace.addLog(); // добавляем второе бревно
    }

    // Альтернативный вариант с проверкой результата
    public boolean addTwoLogsToFireplaceWithCheck(Fireplace fireplace) {
        if (fireplace == null) {
            System.out.println("Ошибка: камин не существует!");
            return false;
        }

        System.out.println(name + " пытается добавить два бревна в камин");

        boolean firstLogAdded = fireplace.addLog();
        boolean secondLogAdded = fireplace.addLog();

        if (firstLogAdded && secondLogAdded) {
            System.out.println("Оба бревна успешно добавлены!");
            return true;
        } else {
```

```
            System.out.println("Не удалось добавить оба бревна");
            return false;
        }
    }
}
```

```
package com.house;

public class Fireplace {
    private int logCount;
    private int maxCapacity;

    public Fireplace() {
        this.logCount = 0;
        this.maxCapacity = 10; // максимальная вместимость камина
    }

    public Fireplace(int maxCapacity) {
        this.logCount = 0;
        this.maxCapacity = maxCapacity;
    }

    // Метод для добавления одного бревна
    public boolean addLog() {
        if (logCount < maxCapacity) {
            logCount++;
            System.out.println("Бревно добавлено. Теперь в камине: " + logCount +
" бревен");
            return true;
        } else {
            System.out.println("Камин полон! Нельзя добавить больше бревен");
            return false;
        }
    }

    // Геттеры для получения информации о состоянии камина
    public int getLogCount() {
        return logCount;
    }

    public int getMaxCapacity() {
        return maxCapacity;
    }

    public boolean isFull() {
        return logCount >= maxCapacity;
    }
}
```

2. Реализация метода от Qwen:

```
package carlson;

import fireplace.Fireplace; // Импорт класса из другого пакета

public class Carlson {
    // Вариант 1: Камин передаётся в метод
    public void addTwoLogsToFireplace(Fireplace fireplace) {
        if (fireplace == null) {
            throw new IllegalArgumentException("Камин не может быть null");
        }
        fireplace.addLogs(2); // или два раза fireplace.addLog();
    }

    // Вариант 2 (альтернатива): Камин хранится как поле
    /*
    private Fireplace fireplace;

    public Carlson(Fireplace fireplace) {
        this.fireplace = fireplace;
    }

    public void addTwoLogsToFireplace() {
        fireplace.addLogs(2);
    }

}

package fireplace;

public class Fireplace {
    private int logs = 0;

    public void addLog() {
        logs++;
        System.out.println("Бревно добавлено в камин. Всего брёвен: " + logs);
    }

    // Или альтернативно:
    public void addLogs(int count) {
        if (count < 0) {
            throw new IllegalArgumentException("Количество брёвен не может быть
отрицательным");
        }
        logs += count;
        System.out.println("Добавлено " + count + " брёвен. Всего: " + logs);
```

```
    }

    public int getLogsCount() {
        return logs;
    }
}
```

Сравнительный анализ кода

Сравнивая код, написанный ИИ-ассистентами, с моей реализацией метода addTwoLogs в классе персонажа Carlson, я заметил следующие ключевые отличия:

И Qwen, и Deepseek не используют для хранения объектов брёвен динамический или статический массив, вместо этого они объявляют в классе камина счётчик брёвен и используют в методе камина addLog просто прибавление единицы к общему числу брёвен. Это сокращает количество кода, но опять же опускает важные детали проекта, потому что каждое бревно - это объект, имеющий свои поля, методы и конструктор. То есть мы просто не получим подробной информации о полене и не сможем обратиться к каждому полену, как к отдельному объекту, несомненно - упущение.

Также в моём коде метод addTwoLogs является более развернутым и позволяет получить больше информации о происходящем и самом сценарии. В нём используется метод Voice, а также выводится состояние камина после добавления в него поленьев, мы можем узнать их количество и то, какими брёвнами он наполнен.

Но стоит отметить и некоторые неплохие доработки кода со стороны ИИ-ассистентов, а именно проверку на наличие камина как объекта и максимальную вместимость у Deepseek, а также генерацию исключения при отсутствии камина и проверку на отрицательное количество брёвен у Qwen.

Конечно качество кода, предоставляемого ИИ-ассистентами зависит только от моего промпта, и если бы я прописал его более подробно, указывая на каждую деталь моего проекта, то скорее всего получил бы идеальный код, который предоставлял тот же функционал, что и моя реализация метода addTwoLogs. Но ИИ-ассистенты могли бы и сразу предложить хороший и полный вариант реализации с возможностью расширения и модификации, чего они не сделали, а значит мой код без сомнений лучше.

Итак, ИИ-ассистенты довольно хорошо и правильно выполнили мои требования, указанные в промпте, но их реализация не сравнится с моей, потому что о своём проекте я знаю гораздо больше чем они.

Выводы по работе

Во время выполнения лабораторной работы №3-4 я научился работать с абстрактными классами, интерфейсами, записями, перечислениями, с динамическими массивами, а именно ArrayList, различными видами исключений, научился переопределять методы toString, hashCode и equals из класса Object, я узнал про анонимные, внутренние и вложенные классы, ознакомился с принципом SOLID и STUPID, узнал как работают проверяемые и непроверяемые исключения, изучил огромное количество теории, пока выполнял подготовку к данному проекту. Углубил свои знания в сфере UML-диаграмм, пока готовил сценарий моего проекта, и, конечно же, придерживался основных принципов ООП во время выполнения данной работы.